ПРАКТИЧЕСКАЯ РАБОТА № 8

ГЕНЕРАЦИЯ ПРОСТЫХ ЧИСЕЛ, ИСПОЛЬЗУЕМЬХ В АСИММЕТРИЧНЬХ СИСТЕМАХ ШИФРОВАНИЯ

Цель работы: Изучение методов генерации простых чисел, используемых в системах шифрования с открытым ключом.

1. Асимметричные системы шифрования

Асимметричные криптосистемы (системы открытого шифрования, с открытым ключом - public key systems) — смысл данных криптосистем состоит в том, что для зашифрования и расшифрования используются разные преобразования. Одно из них — зашифрование — является абсолютно открытым для всех. Другое же — расшифрование — остается секретным за счет секретности ключа расшифрования. Таким образом, любой, кто хочет что-либо зашифровать, пользуется открытым преобразованием. Но расшифровать и прочитать это сможет лишь тот, кто владеет секретным ключом.

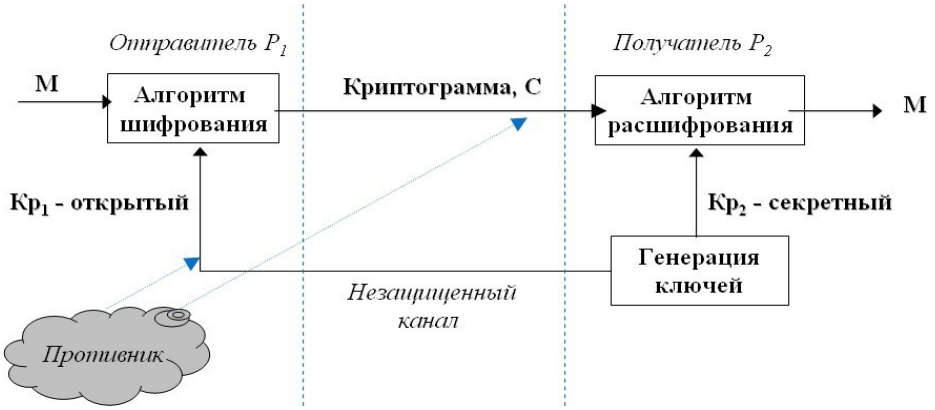

Рис. 1. Обобщенная схема асимметричной криптосистемы

В настоящий момент во многих асимметричных криптосистемах вид преобразования определяется ключом. У пользователя есть два ключа — секретный и открытый. Открытыи ключ публикуется в общедоступном

месте, и каждый, кто захочет послать сообщение этому пользователю — зашифровывает текст открытым ключом. Расшифровать сможет только упомянутый пользователь с секретным ключом. Таким образом, отпадает проблема передачи секретного ключа, как в симметричных системах. Однако, несмотря на все свои преимущества, эти криптосистемы достаточно трудоемки и медлительны. Стойкость асимметричных криптосистем базируется, в основном, на алгоритмической трудности решить за приемлемое время какую-либо задачу. Если злоумышленнику удастся построить такой алгоритм, то дискредитирована будет вся система и все сообщения, зашифрованные с помощью этой системы. В этом состоит главная опасность асимметричных криптосистем в отличие от симметричных.

Алгоритм Диффи-Хеллмана

Алгоритм Диффи-Хеллмана *(Diffie-Hellman)* использует функцию дискретного возведения в степень. Сначала генерируются два больших простых числа n и q. Эти два числа не обязательно хранить в секрете. Далее один из партнеров P_1 генерирует случайное число x и посылает другому участнику будущих обменов P_2 значение

$$A = q^x \bmod n$$

По получении A партнер P_2 генерирует случайное число y и посылает участнику обмена P_1 вычисленное значение

$$B = q^y \bmod n$$

Партнер P_1, получив B, вычисляет $K_x = B^x \bmod n$, а партнер P_2 вычисляет $K_y = A^x \bmod n$. Алгоритм гарантирует, что числа K_y и K_x равны и могут быть использованы в качестве секретного клюпа для шифрования. Ведь даже перехватив числа A и B, трудно вычислить K_y или K_x. Схематично, работа алгоритма Диффи-Хеллмана представлена на рис. 2.

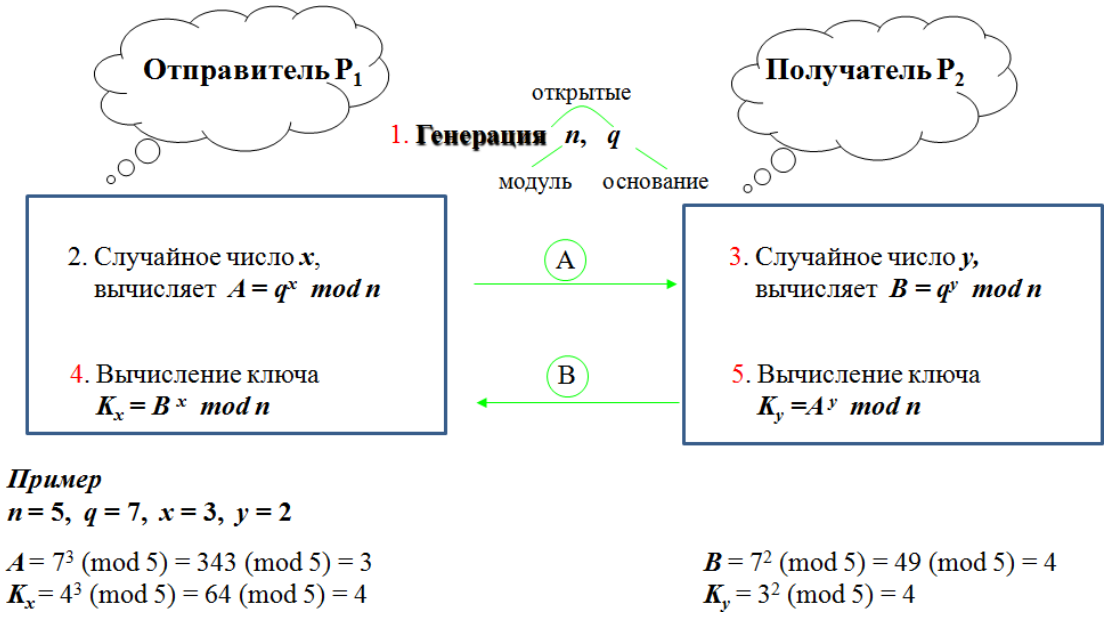

Рис. 2. Алгоритм Диффи-Хеллмана

Алгоритм RSA

Первое практическое воплощение принцип открытого шифрования получил в системе *RSA,* разработанной в 1977 году в Массачусетском Технологическом Институте (США) и получившей свое название от первых букв фамилий авторов: Рональд Ривест *(R.Rivest),* Эди Шамир *(A.Shamir),* Леонард Адлеман *(L.Adleman).*

Идея авторов состояла в том, что взяв целое число N в виде произведения двух больших простых чисел $N=P*Q$, легко подобрать пару чисел Y и X, таких, чтобы для любого целого числа M, меньшего N, было справедливо соотношение:

$$(M^X)^Y = M \bmod N$$

В качестве открытого ключа шифрования в системе *RSA* выступают ключ *Y* и модуль *N,* а секретным ключом для расшифрования сообщений является число *X*.

Процедура шифрования сообщения *M,* рассматриваемого как целое число (такое допущение возможно вследствие того, что любой контент может быть представлен в числовой форме при обработке в средствах вычислительной техники), меньшее *N* (при необходимости длинное сообщение разбивается на отрезки, шифруемые независимо), состоит в вычислении значения:

$$C = M^{Y} mod\ N$$

Расшифрование осуществляется аналогично с использованием секретного ключа

$$M = C^{X} mod\ N$$

Математически строго можно доказать, что определение по паре чисел (*N, Y*) секретного ключа *X*, не проще разложения на простые множители числа *N,* то есть нахождения *P* и *Q*. Задача же разложения на множители целого числа изучается в математике с древнейших времен и известна как сложная вычислительная задача. На настоящий момент разложение числа из нескольких сотен десятичных знаков потребует от современных вычислительных машин сотен лет непрерывной работы.

Схематично, работа алгоритма *RSA* представлена на рис. 3.

Генерация ключей

Получатель 1. P, Q - простые, $N = P \cdot Q$

2. $\varphi(N) = (P-1) \cdot (Q-1)$, $\varphi(N)$ - функция Эйлера

Выбор открытого ключа Y:

$1 < Y \leq \varphi(N)$, НОД$(Y, \varphi(N)) = 1$

Вычисление секретного ключа X:

$X \cdot Y \equiv 1 \pmod{\varphi(N)}$

(N,Y) → *отправителю*

Отправитель **шифрование** M ($M_i = 0, 1, 2, \ldots, N-1$)

3. $C_i = M_i^Y \pmod N$

Получатель **расшифрование** C ($C_1, C_2, \ldots, C_i, \ldots$)

4. $M_i = C_i^X \pmod N$

Пример

Генерация ключей

1. $P = 3, Q = 11, N = P \cdot Q = 33$

2. $\varphi(N) = (P-1) \cdot (Q-1) = 2 \cdot 10 = 20$

$Y = 7$, НОД$(Y, \varphi(N)) = 1$

$X \cdot Y = 1 \pmod{20}$, $7 \cdot 3 = 1 \pmod{20}$, $X = 3$

Сообщение: $M_1M_2 \rightarrow 32$; $M_1 = 3<33$, $M_2 = 2<33$

Шифрование $C_i = M_i^Y \pmod N$

3. $C_1 = 3^7 \bmod 33 = 2187 \bmod 33 = 9$

$C_2 = 2^7 \bmod 33 = 128 \bmod 33 = 29$

Шифротекст 9,29

Расшифрование $M_i = C_i^X \pmod N$

4. $M_1 = 9^3 \bmod 33 = 729 \bmod 33 = 3$

$M_2 = 29^3 \bmod 33 = 24389 \bmod 33 = 2$

Восстановленный текст 3,2

Рис. 3. Алгоритм *RSA*

Методы проверки чисел на простоту

Одна из главных проблем асимметричного шифрования — генерация больших простых чисел. Простейшим методом проверки простоты натурального числа *N* является метод пробных делений: для d = 2, 3, 4 ... мы проверяем выполнение условия $(d, N) > 1$ (здесь (d, N) — наибольший общий делитель чисел *d* и *N*. Число операций, требуемых для этого метода, имеет порядок корня из *N*.

Поэтому уже для чисел порядка $10^{30} \ldots 10^{40}$ он не применим.

В отличие от таких “детерминированных“ тестов существуют еще “вероятностные“ тесты проверки простоты. Для исследуемого числа

проверяется выполнение некоторых, связанных со случайными числами, условий. Если какое- либо из этих условий не выполнено, то N — составное число. Если же все условия выполнены, то с некоторой вероятностью можно утверждать, что N — простое число. Эта вероятность тем ближе к 1, чем большее количество случайных чисел мы проверим.

Обычно эти условия основаны на малой теореме Ферма, утверждающей, что для любого положительного числа b, не превосходящего некоторого простого числа p :

$$b^{(p-1)} = 1(mod\ p).$$

Например, $2^6 = 64 = 63+1 = 1(\mathrm{mod}\ 7)$. Если требуется определить, является ли целое число r простым, то можно выбрать любое положительное целое число b, меньшее r, и проверить, выполнено ли равенство

$$b^{(r-1)} = 1(mod\ r).$$

Если равенство не выполнено, то на основании теоремы Ферма можно быть совершенно уверенным, что r — не простое число. Если же равенство выполнено, то можно лишь предполагать, что r — простое число и поэтому назвать его “псевдопростым по основанию b“. Вероятность $P(x)$ того, что составное число x окажется псевдопростым по случайному основанию, убывает с ростом x.

К сожалению, существуют так называемые числа Кармайкла — такие составные числа, которые обладают свойством:

$b^{(r-1)} = 1(mod\ r)$ для всех b из интервала $[1, r]$, которые взаимно просты с r.

Примером числа Кармайкла является число 561 = 3*11*17.

Классический результат теории чисел — теорема Чебышева — показывает, что доля положительных целых чисел, меньших некоторого целого m и являющихся простыми, близка к $1/(ln\ m)$. Например, доля целых чисел, меньших 10^{100} и являющихся простыми, близка к $1/(1n10^{100}) = 1/230$.

Таким образом, если мы выберем случайно большое целое положительное нечетное число x и будем последовательно проверять на простоту числа x, $x+1, x+2, \ldots$, то, в среднем, мы впервые встретим простое число на шаге с номером $\ln x$.

2. Порядок выполнения практической работы

Для выполнения практической работы необходимо запустить программу *L_PROST.EXE.* На экране дисплея появляется окно, с размещенным в его центре текстовым редактором (для отображения информации об осуществленных программой действиях), в верхней строке окна расположено главное меню, чуть ниже основного меню расположена панель инструментов (для управления быстрыми командными кнопками и другими “горячими” элементами управления), а в самом низу окна, под тестовым редактором, находится строка состояния, в которой указывается подсказка и выводится дополнительная информация. Клавиши панели инструментов, для удобства, снабжены всплывающими подсказками.

Для того чтобы попасть в основное меню, необходимо нажать клавишу F10. Передвижение по главному меню осуществляется клавишами перемещения курсора. Чтобы вызвать пункт меню, нужно нажать клавишу “ENTER”, “ESC” — выход из основного меню.

Для удобства в программе предусмотрена работа с мышью. В этом случае указатель подводится к нужному пункту главного меню или к нужной кнопке на панели инструментов и нажимается левая клавиша мыши, для отказа достаточно нажать клавишу “ESC”.

Кроме основных функций главного меню (“Генерация простого P”, “Поиск в интервале”, “Проверка на простоту” и так далее) панель инструментов содержит клавишу, при нажатии которой выводится информация о программе.

Генерация простого P

Возможность генерации простого числа; количество разрядов генерируемого числа задается пользователем (от 1 до 5).

Поиск в интервале

Возможность поиска простых чисел в заданном интервале. Пользователем задается: начало интервала — значение x, затем длина интервала — значение L. Поиск будет осуществляться в интервале $(x, x+L)$.

При проверке на простоту каждого числа интервала сначала выполняется тест пробных делений на первые по порядку простые числа, а затем проверка по тесту Ферма. Для задания способов проверки на простоту необходимо левой клавишей мыши отметить флажок рядом с названием нужного метода, а затем указать все необходимые данные для начала поиска.

В методе пробных делений исходные данные — количество первых простых писел для деления, а в тесте Ферма надо указать количество оснований и их значения.

По окончании расчета на экран выведутся найденные простыIe числа и их количество. Полную картину результатов работы можно просмотреть в пункте меню “Вывод результатов“.

Проверка на простоту

Возможность проверки на простоту любого числа. Необходимо ввести число и параметры расчета аналогично поиску в интервале.

Вывод результатов

Возможность просмотра всех результатов последней обработки. При входе в программу, когда никаких расчетов еще не производилось,

предоставляется возможность просмотра исходного файла первых простых чисел, используемых для теста пробных делений.

3. Задание

1. Проверить на простоту два произвольных целых писла, разрядностью не менее 5.
2. Распределение простых чисел.

   2.1. Задан интервал вида $[x, x+L]$. Вычислить количество $N(x, L)$ простых чисел в интервале и сравнить с величиной $L/ln(x)$. При каких условиях $П(x,L)/L$ близко к $1/ln(x)$?

   x=2000, L=500,

   – количество простых чисел для деления: 5-15,
   – количество оснований: 1-2.

   2.2. Найти в заданном интервале все простые писла. Пусть $L(i)$ — разность между двумя соседними простыми числами. Построить гистограмму для $L(i)$. Вычислить выборочное среднее Lсред. Сравнить с величиной $ln(x)$, где x — середина интервала.

   Интервал: (1000, 1000+300),

   – количество простых чисел для деления: 5-20,
   – количество оснований: 1-3.

   2.3. Для заданного набора чисел {k} оценить относительную погрешность формулы для k — го простого числа:

   $p(k)=k/ln(k)$, $k = \{10, 15, 20, 30, 35\}$.

3. Методы генерации простых чисел.

   3.1. В заданном интервале построить график относительного количества натуральных писел, проходящих “решето Эратосфена” (т. е. не делящихся на первые k простых).

   Интервал: (500, 500+200).

   Расчет производится для всех k<=10.

3.2. Для заданного интервала:

а) рассчитать точное количество P_t простых чисел в интервале, т.е. при проверке задать только тест на делимость.

Количество первых простых чисел для деления определяется из расчета: максимальное число для деления равно квадратному корню из максимального значения интервала.

6) составить тест с небольшим количеством пробных делений и одним основанием в тесте Ферма. Вычислить количество P_1 вероятно простых чисел, удовлетворяющих этому тесту.

в) составить тест с большим, чем в предыдущем случае, количеством пробных делений и двумя или тремя основаниями в тесте Ферма. Вычислить количество P_2 вероятно простых чисел, удовлетворяющих этому тесту.

Интервал: (1500,1500 + 300).

Проанализировать полученные данные.

3.3. Известно, что в заданном интервале имеются числа Кармайкла. Найти их.

Варианты интервалов: (1050, 1050 + 100)

(1700, 1700 + 100)

(2400, 2400 + 100)

Дополнительные сведения о программе

1. При запуске утилит генерации простого числа, поиска в интервале и проверки на простоту у пользователя запрашивается подтверждение на правильность выбранного метода для работы.
2. Во время работы длительных по исполнению процедур запускается прогресс процесса и гасится окно текстового редактора. По полоске прогресса можно наблюдать и оценивать примерную скорость работы алгоритма и время окончания текущего процесса.
3. Будьте внимательны при установке параметров работы, так как в процессе вычисления по ходу работы эти параметры изменить уже не удастся.

4. Описание "горячих клавиш":

   Ctr1+F1 - ‘Генерация простого P’

   Ctr1+F2 - ‘Поиск в интервале’

   Ctr1+F3 - ‘Проверка на простоту’

   Ctr1+F4 - ‘Вывод результатов’

   Ctr1+X - ‘Выход из программы’

5. В практической работе из-за большого времени счета рекомендуется использовать числа не более пяти разрядов и длину интервала выбирать не более 500, количество оснований для теста Ферма — не более 5.

6. Для правильного функционирования программы, в рабочей директории (вместе с файлом *L_PROST.EXE)* обязательно должны находиться файлы *PROST.TXT* и *WORK.TXT*. Не рекомендуется вносить какие-либо изменения в эти текстовые файлы иначе последствия могут быть непредсказуемые.

Номер варианта	Контрольные вопросы
1,5,7, 3,9,18,28	Почему в качестве первого основания в тестах типа теста Ферма для проверки на простоту очень больших чисел целесообразно использовать число 2?
2,4,6,8, 20,22,24, 26,30	Какова вероятность *P(x)* того, что наугад взятое нечетное очень большое число, не превосходящее *x*, окажется простым?
11,13,15, 10,17,19, 27	Вычислить: 1812 (mod 13), 127 (mod7).
12,14,16, 21,23,25, 29	Сформулируйте суть теста на простоту с использованием пробных делений.